SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Course Code	BCSE301L	**Course Name**	Software Engineering
SLOT	A2+TA2	**Class No.**	VL2025260501406

Assessment Type: **Case Study** | **Total Marks**: 20 | **Weightage**: 10 | **Due Date**: 06-04-2026

Rubrics

Problem understanding & objectives	Clear statement of the case study goal, scope (2-month lab scale), and 2–3 research question	4
Use of literature & references	Relevant existing literature is used correctly, and all claims are supported with proper citations; references are credible and consistent.	4
Methodology & evidence collection	Clear method (data sources, steps followed, metrics/criteria used) and sufficient evidence (repo artifacts, logs, issue samples, experiment notes, etc.).	4
Analysis & findings	Findings are logically derived from evidence; includes limitations/threats to validity and practical implications	4
Report quality & academic integrity	**Well-structured, readable document with figures/tables; Plagiarism + AI-generated text in the project report must be <10%**	4

Instructions to Student

1. Complete the assigned case studies within the schedule announced and upload all deliverables on VTOP within the deadline
2. Deliverables include – A complete case study report including complete source code if developed prototype for validation and test.
3. No more than four students are allowed to choose the same topic.
4. This is an individual activity; any similarity found between submitted documents will result in a penalty.

List of Case Studies

Case Study 1

Using existing LLM4SE literature as baseline, study how 3 prompting strategies (zero-shot, context-enriched, iterative "generate→run→repair") affect unit test executability and fault detection on one small open-source module; validate with a minimal test runner and a mutation-score proxy.

Case Study 2

Based on empirical SE guidance on evaluating LLM usage, analyze a set of student tasks and quantify "verification debt" (code accepted without running tests/linters); optionally validate by instrumenting a tiny logging script that records when tests were executed before commit.

Case Study 3

Using published privacy engineering guidance that privacy must be considered across the entire SDLC, select a small app scenario (student app handling personal data) and perform a phase-wise privacy activity mapping (plan/design/build/operate), then validate by creating a minimal data-flow diagram + a small threat checklist.

Case Study 4

Conduct a comparative case study of security threat modeling (SSDLC mindset) vs privacy threat/risk modeling for the same small system; use existing privacy-by-design literature to identify what risks are missed if teams only do security threat modeling; validate with a small "risk register" template filled for one scenario.

Case Study 5

Using DevSecOps trend literature, analyze which CI security gates (SAST, dependency scan, secret scan) prevent the most realistic issues in a toy repo and which produce false positives/friction; optionally validate by running open tools on a curated set of commits and reporting signal-to-noise.

Case Study 6

Study how "observability readiness" (structured logs, metrics, trace IDs, dashboards) changes release confidence and incident diagnosis time in a small service; base the framework on observability/SRE trend literature and validate with a tiny prototype that emits logs/metrics and a short incident drill.

Case Study 7

Using AIOps trend descriptions (real-time monitoring, predictive analytics, automated remediation), document how a small team could implement a minimal triage workflow (correlate alerts with deploys, rank probable causes); optionally validate with a scripted "incident replay" using logs from a toy service.

Case Study 8

Based on current discussions that SDLC is being reshaped by AI and security must evolve accordingly, analyse a repository where AI-generated code was used (or simulate with a small snippet set) and classify risks (insecure defaults, licensing ambiguity, missing validation); validate via a small checklist and sample review outcomes.

Case Study 9

From an issue tracker of a real open-source project, extract ambiguous requirements and apply literature-backed rewriting patterns (e.g., add measurable acceptance criteria); optionally validate by mapping rewritten requirements to a small set of acceptance tests (even pseudo-tests) and assessing traceability completeness.

Case Study 10

Using DevSecOps/automation trends, propose a minimal policy-as-code rule set for student repos (tests must pass, secrets banned, dependency vulnerabilities threshold, required docs) and evaluate it on 3 public repos; optionally validate by implementing 2–3 simple CI checks and reporting how many violations are caught.

Case Study 11

For a selected open-source module, identify a set of refactorings (rename, extract method, move method/class) and study how refactoring frequency correlates with short-term defect introduction (bug-fix commits after refactors); validate with commit history mining and a small manual sample review.

Case Study 12

Apply technical debt/hotspot identification techniques (complexity, churn, code smells) to rank top risk files; validate by checking whether these files contain more bugs/bug-fixes historically and propose a prioritized remediation plan.

Case Study 13

In a real project repo (issues + commits + PRs), assess how well user stories can be traced to code changes and tests; identify common traceability gaps (missing IDs, mixed commits) and propose a lightweight traceability policy.

Case Study 14

Analyze a repo's CI failures across a time window and categorize root causes (flaky tests, environment drift, dependency updates, non-determinism); validate with pipeline logs and propose a "stabilization checklist" to reduce failure recurrence.

Case Study 15

Identify flaky tests in a chosen project (tests that pass/fail without code changes) and classify causes (timing, ordering, network, randomness); validate with repeated test runs and recommend containment (quarantine, retries, determinism fixes).

Case Study 16

Track changes in a library/service API over versions and assess compatibility breaks using semantic versioning principles; validate by mapping changes to version increments and reporting mismatches, then propose an improved versioning policy.

Case Study 17

Build a defect taxonomy from 50–100 issues of a real project (requirements defects, logic errors, UI, concurrency, performance); validate with inter-rater agreement (2 reviewers label a subset) and propose prevention actions per defect class.

Case Study 18

Measure how review size (LOC changed), number of reviewers, and time-to-merge relate to post-merge defects; validate by mining PR metadata and linking to subsequent bug-fix commits, then propose review-size or checklist guidelines.

Case Study 19

Define intended architecture rules (layering, module boundaries) and evaluate erosion over time (new forbidden dependencies, cycles); validate using static dependency analysis and propose refactoring steps to restore modularity.

Case Study 20

For a project with recorded estimates (story points, planned hours), study estimation error distribution and identify factors that drive error (unclear requirements, dependency work, refactoring); validate using sprint records or issue metadata and propose an estimation improvement protocol.

HEADINGS TO BE INCLUDED IN CASE STUDY REPORT

1. **Aim**
2. **Background Literature**
3. **Research Questions**
4. **Data Sources**
5. **Method**
6. **Metrics**
7. **Threats to Validity / Limitations**
8. **Expected Outcome**
9. **References**